



One-Venue DEX Settlement Stateless Conduit, Syn- chronous Atomic Carry- Fills, and Post-Quantum Cross-Network Transport

Zach Kelling

Lux Industries

June 2026

Abstract

The Lux decentralized exchange is *one venue*—the D-Chain, a virtual machine that runs a central-limit order book as a consensus-native operation (LP-032)—reached from every EVM chain through *one settlement primitive*, the 0x9999 precompile (LP-9999). This paper specifies and proves correct the settlement *layer* that composes them. We make the 0x9999 precompile a *stateless conduit* that holds no order book and no ledger: the single book and the single ledger live on the venue. We show that settlement on the Lux primary network is synchronous within a single block—preserving the Uniswap-V4 `swap()` interface that returns a balance delta in the same transaction—via atomic shared-memory *carry-fills*, and we prove this construction is consensus-safe whereas the naïve per-validator venue call is not. We show that settlement across networks is asynchronous via warp/ICM, with the venue’s own post-quantum consensus quorum certificate—not a bespoke exchange certificate—as the sole cross-network trust anchor, and we prove the cross-network path adds zero trust surface beyond the venue’s existing validator set. Finally, we show that removing state from the conduit dissolves an empty-account reaping hazard that otherwise strands funds on every EVM chain, and that one venue makes LUX a universal cross-chain fee currency with shared, unfragmented liquidity.

Contents

1	Introduction	2
2	The empty-account reaping hazard	3
3	The decomplicated architecture	4
3.1	Values, venue, conduit, transport	4
4	Synchronous settlement on the primary network	4
4.1	Carry-fills	4
4.2	Developer interface	5
5	Asynchronous settlement across networks	5
5.1	The acceptance predicate and zero new trust	5
5.2	Post-quantum certificate	6
6	The stateless conduit	6
7	Universal LUX fee rail	7
8	Implementation status	7
9	Related work	7

1 Introduction

A decentralized exchange must answer three questions: *where do orders match*, *where does custody live*, and *how does value move across chain boundaries*. The Lux design answers the first two with a single object—the D-Chain venue (LP-032)—and the third with a single primitive—the 0x9999 settlement precompile (LP-9999). The discipline that keeps the system simple is that these answers stay separate: matching and custody are the venue’s; movement is the conduit’s; and the only thing that varies between chains is the transport that carries intents to the venue and fills back.

That discipline had drifted. The `0x9999` precompile accumulated an in-trie matching engine and a per-account ledger inside its own account storage, becoming a second venue beside the D-Chain venue. The drift was not merely inelegant: it manifested as a reproducible, fund-stranding production failure (Section 2) that blocked every real fill on every EVM chain. The failure is the symptom; the disease is the braiding of *where state lives* into the settlement adapter. This paper decomplexes them and proves the resulting system correct.

Contributions.

1. A stateless-conduit settlement architecture (Section 3) in which the venue is the unique holder of the order book and ledger, the conduit holds only escrow, and transport is the single orthogonal axis.
2. A consensus-safety result for synchronous in-block settlement (Section 4): the atomic carry-fills construction yields deterministic replay (Theorem 4.1), while the per-validator venue call admits a safety violation (Theorem 4.2).
3. A zero-new-trust result for cross-network settlement (Section 5): a cross-network fill is accepted exactly when a $\geq \frac{2}{3}$ -of-venue-stake certificate validates it (Theorem 5.1), reusing the venue’s post-quantum consensus certificate rather than a bespoke exchange certificate.
4. A reaping-immunity result (Theorem 6.1) and a universal-fee-rail construction (Section 7).

2 The empty-account reaping hazard

The `0x9999` precompile is *always-on*: it is dispatchable by its address whether or not a state account object exists there. When the precompile owns a ledger, it writes that ledger into its own account’s storage. We recall the Ethereum emptiness predicate.

Definition 2.1 (EIP-158/161 emptiness). *A state account is empty iff $\text{nonce} = 0 \wedge \text{balance} = 0 \wedge \text{codeHash} = \text{keccak256}(\varepsilon)$. Storage contents are not part of the predicate.*

The per-transaction finalization step `Finalise(deleteEmptyObjects = true)` deletes every empty account *together with all of its storage*. An ERC-20 deposit credits the token contract’s storage (so `balanceOf(token, 0x9999)` rises) and writes the precompile’s reserve and credit rows into `0x9999`’s storage—leaving `0x9999`’s own account empty (no balance, no nonce, no code). Finalization then reaps it.

A real-StateDB probe matrix (geth host, EIP-158 enabled) isolates the discriminator: the empty account, not the nested token call.

deposit nested call shape	0x9999 non-empty	EIP-158	ledger survives Finalise(true)
native (adds balance)	yes	yes	on yes
ERC-20 (token storage only)	yes	no	on no (reaped)
ERC-20, no nested call	no	no	on no (reaped)
ERC-20	yes	no	off yes

Table 1: Empty-account reaping isolated. Native deposits survive because the balance credit makes 0x9999 non-empty; ERC-20 deposits do not, with or without a nested call; disabling EIP-158 prevents reaping. The discriminator is emptiness, not the nested call.

Consequence. No market can be funded, so no fill ever settles ($DEXFill$ count = 0 on every EVM chain), and any ERC-20 deposited to 0x9999 is stranded: its claim row is reaped while the tokens remain in the token contract. Because 0x9999 is always-on on every Lux EVM past the activation fork, the exposure is latent on every chain.

Why the marker is the wrong fix. Forcing $nonce(0x9999) = 1$ keeps the account non-empty and suppresses the symptom, but it preserves the disease: a precompile owning a ledger, now defended against the host. The correct remedy is to remove the state from the conduit. Section 3 does so; Theorem 6.1 shows the hazard then cannot arise.

3 The decomplexed architecture

3.1 Values, venue, conduit, transport

Definition 3.1 (Settlement values). *An order intent is the tuple a trader submits: market, side, limit, size, and an escrow reference; it is content-addressed and chain-agnostic. A fill receipt is the tuple the venue produces: the matched legs as per-leg asset deltas $\langle rail, owner, asset, amount, spent \rangle$ together with a quorum certificate; it is self-authenticating and chain-agnostic.*

Definition 3.2 (Venue). *The venue is the D-Chain (LP-032): the unique holder of the order book (order:, book:, market:, trade:), the ledger (balance:, locked:), and the seam-escrow record (seamintent:, seamout:). Matching is a VM-native operation at block execution, deterministic and parallel across markets.*

Definition 3.3 (Conduit). *The conduit is the 0x9999 precompile, identical on every EVM chain. It performs exactly: escrow the physical asset; forward an intent to the venue; apply a fill the venue produced; release escrow. It holds no order book, reserve, or credit ledger, and writes no durable row to its own account storage (Section 6). It does not branch on the transport.*

Definition 3.4 (Transport). *A transport provides $\text{SUBMIT}(\text{intent}) \rightarrow \text{ref}$ (forward $C \rightarrow D$, revert-safe) and $\text{APPLY}(\text{ref}) \rightarrow \text{receipt}$ (deliver $D \rightarrow C$, value-bound, exactly-once). Two implementations exist: atomic (same network as the venue; synchronous) and warp (a separate network; asynchronous). The venue and the conduit never branch on which.*

A new chain—L1, L2, or L3—adds no settlement code; it binds a transport. This is the sense in which all chains “work the same.”

4 Synchronous settlement on the primary network

On the Lux primary network the C-Chain, the D-Chain venue, and any co-resident L1/L2/L3 execute inside a single `luxd` process with atomic shared memory between chains. A swap therefore settles synchronously within one block and returns a balance delta from `swap()`, preserving the Uniswap-V4 interface.

4.1 Carry-fills

Definition 4.1 (Carry-fills). *The block proposer computes the match once against the venue’s consensus-accepted state and records the resulting fill in the block via atomic shared memory. Validators do not re-run the match; they replay the carried fill from the block bytes and verify consistency with the venue accepted state they also hold. The C-Chain block and the corresponding venue state transition are co-committed within the single-proposer block.*

We model a block’s settlement as a function of two inputs: the block bytes b and the venue’s accepted state σ_D at the block’s parent. Write $\text{Settle}(b, \sigma_D)$ for the post-state a validator computes.

Theorem 4.1 (Synchronous determinism). *Under carry-fills (Definition 4.1) with (C-S1) a single proposer-computed match, (C-S2) settlement reading only the venue’s accepted state—never live, wall-clock, stream-ordered, or non-deterministically-accelerated state—and (C-S3) no per-validator venue query during verification, any two honest validators that agree on b and on σ_D compute identical post-states: $\text{Settle}(b, \sigma_D) = \text{Settle}(b, \sigma_D)$ byte-for-byte.*

Proof. The settlement-relevant inputs are exactly b and σ_D . The block bytes b are agreed by gossip and consensus on the C-Chain block; σ_D is agreed by consensus on the venue (both chains being co-resident and at accepted state). By (C-S2) `Settle` reads no other input; by (C-S1) the carried fill in b is the unique match, not a per-validator recomputation; by (C-S3) no validator introduces an independent venue read. Hence `Settle` is a pure function of the two agreed inputs, and pure functions of equal inputs return equal outputs. A GPU accelerator is admitted only when known-answer-tested byte-identical to the reference matcher, so it does not enlarge the input set. $\square$ $\square$

Theorem 4.2 (The per-validator venue call forks). *Consider the alternative construction in which each validator, during verification of a swap transaction, independently queries the venue for a match. There is a reachable execution in which two honest validators return different match results and therefore disagree on the block’s validity—a safety violation.*

Proof (by the existing adversarial witness). Let two validators verify the same swap against venue states that differ in the transient ordering of a concurrently-relayed order—a difference invisible to consensus because the venue query is not a function of agreed state alone. One validator matches against a book containing the concurrent order and returns fill f_1 ; the other matches against a book without it and returns $f_2 \neq f_1$. The two validators then hold different post-states for the same block and vote incompatibly. This execution is realized by the D-Chain proxy’s adversarial test `TestRED_PerValidatorRelay_SplitsConsensus`, which exhibits a concrete split. The degree of freedom is the per-validator venue read; Theorem 4.1 removes it. $\square$ $\square$

Theorems 4.1 and 4.2 together are the standard’s mandate: synchronous settlement *must* be carry-fills, never a Verify-time venue call.

4.2 Developer interface

`swap()` keeps its V4 signature and returns a balance delta in the same transaction—no intent acknowledgement, no ABI change, no second call. Applications on any primary-network chain integrate the DEX as they would a local pool.

5 Asynchronous settlement across networks

A chain on a separate network—its own network identifier, no atomic shared memory with the venue—settles across the boundary via warp/ICM. Settlement is asynchronous: the conduit escrows the input and forwards the intent; the fill arrives later carrying the venue’s certificate. The V4 call site is unchanged; the realized fill is read from the D→C settlement.

5.1 The acceptance predicate and zero new trust

Let $V_D(h)$ be the venue’s canonical validator set at P-Chain height h , with stake weights, and let a certificate π over a message M at height h *validate* iff the signing weight is $\geq \frac{2}{3}$ of $\text{totalWeight}(V_D(h))$ under the registry-pinned scheme.

Theorem 5.1 (Cross-network zero-new-trust). *The conduit accepts a cross-network fill receipt r iff (1) r is the payload of a warp message M whose certificate π validates against $V_D(h)$ with quorum read from configuration (never from M); (2) $M.\text{sourceChainID}$ is the venue and $M.\text{networkID}$ is local; (3) the reference is unconsumed and is marked consumed before credit; and (4) the decoded receipt satisfies the settlement binds. Under these rules, an adversary that controls the transport (the relayer) and $< \frac{1}{3}$ of the venue’s stake cannot cause acceptance of any fill the venue did not produce. The cross-network trust surface equals V_D .*

Proof. Acceptance requires a validating π (1). By the unforgeability of the aggregate scheme and quorum being read from configuration with a hard $\geq \frac{2}{3}$ floor, producing a validating π requires signatures of $\geq \frac{2}{3}$ of $V_D(h)$ ’s weight. An adversary with $< \frac{1}{3}$ of venue stake cannot reach that threshold even colluding with all non-venue parties, since the floor is on venue weight and is not attacker-settable. The relayer’s control is confined to delivery: it cannot alter the payload (the certificate covers the message bytes), cannot replay (3), and cannot redirect (2). Hence acceptance implies $\geq \frac{2}{3}$ of venue stake attested r , i.e. the venue produced it. The only trust assumption invoked is honesty of $\geq \frac{2}{3}$ of V_D —which is exactly the trust required to trade on the venue at all. Therefore no trust beyond V_D is added. $\square$ $\square$

No bespoke exchange certificate. A DEX-specific aggregate with a verifier registry was prototyped and deprecated after adversarial review found a value-blind certificate rebind and an attacker-settable wire quorum collapsing to a single signer. The standard reuses the consensus certificate and never mints a second trust mechanism; condition (1) above pins quorum to configuration precisely to exclude the wire-quorum attack.

5.2 Post-quantum certificate

The venue certificate is the consensus quorum certificate, post-quantum under LP-020/LP-073. The primary lane is a Corona (Ring-LWE) threshold group signature whose validity structurally implies the $\geq \frac{2}{3}$ floor and which reuses the venue’s existing consensus group key (no new exchange DKG, hence no trusted-dealer reintroduction). The fallback lane is an ML-DSA (FIPS-204) certificate set summed to $\geq \frac{2}{3}$ stake at verification, requiring no DKG. A `certType` registry

selects classical, Corona, or ML-DSA with no ABI change, profile-gated so classical and post-quantum compose rather than braid. Both lanes verify deterministically and inline-safely; no non-determinism enters the certificate check, so Theorem 4.1 extends to the cross-network apply.

6 The stateless conduit

The conduit exposes four operations, each escrow-then-transport, with no durable write to its own account storage:

operation	effect
deposit	escrow in (native balance, or ERC-20 into the token’s trie) $\rightarrow$ SUBMIT(credit)
swap	escrow input leg $\rightarrow$ SUBMIT(order); same-network: receive the carried fill and <i>return the balance delta</i> ; cross-network: deliver the V4 delta when the fill lands
fill	APPLY(ref) $\rightarrow$ release escrow to the bound owner (value-bound, exactly-once)
withdraw	SUBMIT(withdraw) $\rightarrow$ venue debits its ledger $\rightarrow$ D $\rightarrow$ C release $\rightarrow$ fill

Theorem 6.1 (Reaping immunity). *If the conduit’s only state mutations per transaction are (a) its native balance, (b) writes to a token contract’s storage, (c) host transport operations on the atomic backend, and (d) a transaction-scoped reentrancy guard in transient storage, then **Finalise**(deleteEmptyObjects = **true**) over the conduit account neither loses settlement state nor strands escrow.*

Proof. No persistent **SetState** to the conduit account survives the transaction: (a) is the account balance, not storage; (b) is another account’s storage; (c) is the host atomic backend, a distinct account; (d) is transient storage, discarded at transaction end and never trie-resident. Hence after the transaction the conduit account has no own storage rows. If the account is empty it is reaped, but reaping removes only an account with no balance, nonce, code, or storage—so nothing of value is lost; the ERC-20 escrow is the token’s storage (untouched by reaping the conduit), and the claim on it is a venue ledger row (not on the conduit at all). The precompile remains dispatchable by address regardless of account existence. Therefore no settlement state is lost and no escrow is stranded. $\square$ $\square$

The nonce marker is thus unnecessary and is removed. Custody follows: the tokens are a fact (the token’s storage), the claim is a venue ledger row, and conservation reduces to $\sum_{asset} \text{escrow}(asset) \geq \sum_{asset} \text{venue-backed-releasable}(asset)$, enforced fail-closed by the venue’s debit and by the value-bound conduit release.

7 Universal LUX fee rail

One venue makes LUX a universal cross-chain fee currency with shared liquidity.

Proposition 7.1 (Fee-rail liveness). *If every chain lists initial $\langle \text{native}, \text{LUX} \rangle$ liquidity in the venue at genesis, then every transaction can pay a LUX-denominated fee from the chain’s native asset by a single conduit swap, atomic with the transaction, synchronous on the primary network.*

Proof. By hypothesis a $\langle \text{native}, \text{LUX} \rangle$ market exists in the venue from block zero, so the venue can match a *native* $\rightarrow$ LUX order of the fee size against genesis liquidity. The conduit escrows the native fee input and forwards the order; on the primary network the fill is carried in the same block (Definition 4.1), so the LUX proceeds are available within the transaction to pay the fee. The swap and the fee debit are committed together with the transaction. $\square$ $\square$

Because all chains settle to the same book, $\langle native, LUX \rangle$ liquidity is shared rather than fragmented per chain—the property a per-chain in-trie book cannot provide, and a further reason the second book is removed.

8 Implementation status

The venue (LP-032) and the always-on conduit are live on mainnet and testnet; the D-Chain matcher is bootstrapped and serving. No real fill has settled to date because of the reaping hazard of Section 2, which is reproduced on a real **StateDB** harness. The stateless-conduit de-complect, the carry-fills synchronous path, the warp cross-network transport, the post-quantum certificate gating, and the fee rail are specified here and in LP-311; the consensus post-quantum certificate lanes (Corona, ML-DSA) and the D-Chain proxy’s atomic relay primitive exist. Conformance is reached when a real cross-asset fill settles—synchronously on the primary network, asynchronously across networks—with the conduit holding zero reapable storage, the venue holding the sole book and ledger, the certificate as the sole cross-network trust, and the nonce marker absent.

9 Related work

LP-032 specifies the D-Chain venue and the fill receipt. LP-9999 specifies the **0x9999** precompile ABI. LP-007 and LP-008 specify CLOB matching and multi-asset trading; LP-040/041/042 specify AMM liquidity sources (not matchers). LP-020 and LP-073 specify the Quasar consensus certificate and Pulsar post-quantum finality reused for cross-network settlement. LP-201 specifies the ZAP universal wire used for the receipt encoding. LP-103 and LP-104 specify the explorer and graph engine that index fills from the venue’s trade rows.

References

- [1] Lux Industries, “LP-032: DEX VM,” Lux Proposals.
- [2] Lux Industries, “LP-9999: **0x9999** Receipt-Settlement Precompile,” Lux Proposals.
- [3] Z. Kelling, “LP-311: One-Venue DEX Settlement,” Lux Proposals, 2026.
- [4] Lux Industries, “LP-020: Quasar Consensus Certificate,” Lux Proposals.
- [5] Lux Industries, “LP-073: Pulsar Post-Quantum Finality,” Lux Proposals.
- [6] V. Buterin, “EIP-158: State clearing,” Ethereum Improvement Proposals.